

Category C — Practice Problems

Problem 1

The Piggy Bank

Scenario

A savings app tracks how much money a kid has put away.

Problem Statement

Create a PiggyBank class where money can only be added or removed through specific actions — never set directly to any amount.

Requirements

- The savings amount must be private, changed only by deposit and withdraw methods.
- A withdrawal larger than the current savings must be rejected, not applied.
- Give the piggy bank a final ID that's fixed the moment it's created.
- Provide a way to check the current savings, but no way to set it directly.

Expected Behavior

- A new piggy bank starts at 0 savings.
- Depositing adds exactly that amount.
- Withdrawing more than what's saved leaves the amount unchanged.
- There's no method anywhere that lets you set the savings to a specific number directly.

Sample Input/Output

```
PiggyBank pb = new PiggyBank("PB-1");  
pb.deposit(100) -> savings = 100  
pb.withdraw(30) -> savings = 70  
pb.withdraw(500) -> rejected, savings stays 70
```

Hints

Hint 1: Ask yourself: if this field were public, what's the worst thing someone could do to it? That answer tells you it needs to be private.

Hint 2: One keyword locks the ID in place forever, right from the constructor.

Problem 2

The Quiz Scorecard

Scenario

A quiz app records whether each answer you gave was right or wrong.

Problem Statement

Design a Scorecard class that stores each answer's result privately, and reveals only the final score — never the raw list of right/wrong answers.

Requirements

- Store the results (true for correct, false for incorrect) in a private array, filled in one answer at a time.
- Provide a method to record the next answer's result.
- Expose only the total score (count of correct answers) — never the array itself, in any form.
- The total number of questions must be fixed when the scorecard is created.

Expected Behavior

- Recording correct, correct, wrong, correct gives a score of 3.
- There's no getter that returns the array of results, directly or indirectly.
- Trying to record more answers than the fixed question count should be ignored or rejected.

Sample Input/Output

```
Scorecard sc = new Scorecard(4);  
sc.recordAnswer(true); sc.recordAnswer(true); sc.recordAnswer(false); sc.recordAnswer(true);  
sc.getScore() -> 3
```

Hints

Hint 1: The array only needs to exist inside the class — nothing outside ever needs to see it, only a number computed from it.

Hint 2: Keep a separate counter for "how many answers recorded so far" so you know where the next one goes in the array.

Problem 3

The Nickname Tag

Scenario

A chat app shows a friendly short nickname instead of your full name.

Problem Statement

Create an immutable NameTag class that takes a full name once and builds a nickname from it — first name plus the last name's initial.

Requirements

- Take one full name string in the constructor (e.g., "Maria Gomez") and split it into first and last name.
- Store whatever you keep as final fields — nothing about the name should be changeable after creation.
- Provide a method that returns the nickname (e.g., "Maria G."), built from the stored parts.
- Assume the full name always has exactly one first name and one last name, separated by a single space.

Expected Behavior

- `new NameTag("Maria Gomez").getNickname()` returns "Maria G."
- Once created, there is no method that changes the stored name in any way.
- Two NameTag objects built from the same full name behave identically but remain separate objects.

Sample Input/Output

```
NameTag tag = new NameTag("Maria Gomez");  
tag.getNickname() -> "Maria G."
```

Hints

Hint 1: String has a method that splits text apart wherever a space appears — that's exactly what you need in the constructor.

Hint 2: Do all your string splitting once, inside the constructor, and store the results — don't redo it every time `getNickname()` is called.

Problem 4

The Locker Code

Scenario

A gym locker has a combination code that members can change.

Problem Statement

Design a Locker class where the combination can be changed, but never read back directly from outside the class.

Requirements

- The combination code must be private, with no getter at all.
- Provide a method to change the code that requires the current code to be entered correctly first.
- If the wrong current code is given, the change must be rejected and the code must stay the same.
- Give the locker a final locker number, fixed at creation.

Expected Behavior

- Changing the code with the correct current code succeeds.
- Changing the code with the wrong current code leaves the combination unchanged.
- There is no way, from outside the class, to read the current code.

Sample Input/Output

```
Locker l = new Locker(101, "1234");
```

```
l.changeCode("1234", "5678") -> success
```

```
l.changeCode("0000", "9999") -> rejected, code is still "5678"
```

Hints

Hint 1: "Write-only" just means: a method to set it exists, but no method to read it exists — don't add a getter out of habit.

Hint 2: Check the old code first, before touching the field at all — if the check fails, the method should do nothing else.

Problem 5

The Attendance Sheet

Scenario

A teacher marks which students are present in class today.

Problem Statement

Design an AttendanceSheet class that stores present students internally but only reveals a count and a yes/no lookup — never the full list.

Requirements

- Store the names of present students in a private array (fixed size is fine — assume a maximum class size).
- Provide a method to mark a student present.
- Provide a method that returns how many students are present, and another that checks whether one specific name is present.
- There should be no method that returns the whole array of names.

Expected Behavior

- Marking "Ana", "Ben", and "Ana" again present results in a count of 2 (no duplicates).
- isPresent("Ben") returns true; isPresent("Chen") returns false.
- There's no way to retrieve the full list of present students directly.

Sample Input/Output

```
AttendanceSheet sheet = new AttendanceSheet(30);
sheet.markPresent("Ana"); sheet.markPresent("Ben"); sheet.markPresent("Ana");
sheet.getPresentCount() -> 2
sheet.isPresent("Ben") -> true
```

Hints

Hint 1: Before adding a name, check whether it's already in the array — that's what avoids counting the same student twice.

Hint 2: getPresentCount() and isPresent() only need to loop through the private array internally — they don't need to expose it to do that.